

Lab 9: Sensor Simulation for Autonomy

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

1	Overview	2
2	Learning Objectives	2
3	Pre-Lab: Quick Review	2
3.1	LiDAR data	3
3.2	Occupancy grid	3
3.3	World grid indices	3
3.4	Pre-Lab Questions	3
4	Part 1: Simulating a 2D LiDAR (30 min)	4
4.1	Step 1.1 — Define a simple world	4
4.2	Step 1.2 — Ray-segment intersection	4
4.3	Step 1.3 — Cast one ray against all segments	4
4.4	Step 1.4 — Full 360° scan	4
5	Part 2: From One Scan to an Occupancy Grid (30 min)	5
5.1	Step 2.1 — World grid helpers	5
5.2	Step 2.2 — Rasterize one ray	5
5.3	Step 2.3 — Update the grid with one full scan	5
5.4	Step 2.4 — Short-answer question	5
6	Part 3: Building the Map from Multiple Poses (30 min)	6
6.1	Step 3.1 — Drive the robot	6
6.2	Step 3.2 — Final map	6
6.3	Step 3.3 — Effect of noise	6
6.4	Step 3.4 — Discussion question	6
7	Part 4: A Taste of Vision with OpenCV (30 min)	6
7.1	Step 4.1 — Load and inspect	7
7.2	Step 4.2 — Convert to HSV	7
7.3	Step 4.3 — Find the cone	7
7.4	Step 4.4 — Convert a pixel to a steering decision	7

7.5	Step 4.5 — Compare with LiDAR	7
8	Wrap-Up	7
9	Reference	8

1 Overview

In this lab you build the perception half of the **sense–plan–act** loop from the Week 10 lecture. Starting from an empty 2D world of walls and obstacles, you will simulate a 2D LiDAR via ray casting, convert its returns into a world-frame point cloud, and fuse many scans into a single **log-odds occupancy grid**. A short OpenCV exercise at the end closes the loop by contrasting range sensing with appearance-based sensing on a synthetic “camera” image.

By the end of the lab you should have a working mini-perception stack that takes robot poses in, sensor measurements out, and a map of the environment as its product — exactly the representation that next week’s path planners (Dijkstra, A*) will consume.

Duration: 2 hours

Software required:

- Python environment from Lab 0
- Libraries: `numpy`, `matplotlib`, `opencv-python`, `scikit-image` (the Colab link will `pip install` if needed)

2 Learning Objectives

By the end of this lab, you will be able to:

1. Simulate a 2D LiDAR by casting rays against line-segment obstacles and returning the closest hit per bearing.
2. Convert a scan from polar (r, α) to Cartesian (x, y) in both the sensor frame and the world frame.
3. Rasterize a ray into grid cells using Bresenham’s algorithm (`skimage.draw.line`).
4. Apply the log-odds update rule with `l_free = -0.4` and `l_occ = +0.85` to fuse many scans into a probabilistic occupancy grid.
5. Run a basic OpenCV pipeline — BGR→HSV, color thresholding, contour detection — on a synthetic image and extract an object’s pixel location.

3 Pre-Lab: Quick Review

Complete this section **before** coming to lab.

3.1 LiDAR data

A 2D LiDAR returns an array of N ranges at known bearings: $\{(r_0, \alpha_0), (r_1, \alpha_1), \dots, (r_{N-1}, \alpha_{N-1})\}$.

Polar $\rightarrow$ Cartesian in the **sensor frame**:

$$x_i^S = r_i \cos(\alpha_i), \quad y_i^S = r_i \sin(\alpha_i).$$

Sensor $\rightarrow$ world (robot pose (x_R, y_R, θ)):

$$\begin{bmatrix} x_i^W \\ y_i^W \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_i^S \\ y_i^S \end{bmatrix} + \begin{bmatrix} x_R \\ y_R \end{bmatrix}.$$

3.2 Occupancy grid

Each cell (i, j) stores a **log-odds** value l_{ij} :

- $l = 0$ means unknown ($p = 0.5$).
- $l < 0$ means likely free; $l > 0$ means likely occupied.
- Update rule for one LiDAR ray: add $l_{\text{free}} = -0.4$ to every cell the beam passes through, and $l_{\text{occ}} = +0.85$ to the endpoint cell.
- Clamp $l \in [-5, +5]$ to prevent saturation.

Convert back to probability with $p = 1/(1 + e^{-l})$.

3.3 World grid indices

With grid origin $(x_{\min}, y_{\min})$ and cell size **res**:

$$i = \left\lfloor \frac{x - x_{\min}}{\text{res}} \right\rfloor, \quad j = \left\lfloor \frac{y - y_{\min}}{\text{res}} \right\rfloor.$$

3.4 Pre-Lab Questions

Answer before lab (will be collected at the top of the notebook):

1. A LiDAR has 360 rays spanning 360° . The beam at index $i = 90$ returns $r = 5$ m. Using the sensor-frame convention above, what are the Cartesian coordinates of that point? (Assume rays start at 0° on the $+x$ axis and step in $+1^\circ$ increments.)
2. A cell starts at $l = 0$ and is then observed as **free** three times and **occupied** once with the standard values $l_{\text{free}} = -0.4$ and $l_{\text{occ}} = +0.85$. What are the final log-odds and the corresponding probability of occupancy?
3. Why is HSV a better color space than BGR for detecting a red traffic cone under varying lighting? One sentence.

4 Part 1: Simulating a 2D LiDAR (30 min)

4.1 Step 1.1 — Define a simple world

The world is a list of line segments (each segment is a pair of endpoints $((x_1, y_1), (x_2, y_2))$). Use the starter map provided in the notebook: a 10 m $\times$ 10 m room with an L-shaped internal obstacle.

Plot all segments with `plt.plot` and add a red dot for the robot at its initial pose $(x_R, y_R, \theta) = (2.0, 2.0, 0.0)$.

4.2 Step 1.2 — Ray-segment intersection

Implement the function:

```
def ray_segment_intersection(origin, direction, seg):
    """Return the distance along `direction` from `origin` to segment `seg`,
    or np.inf if they do not intersect in front of the origin.
    origin      -- (x, y)
    direction    -- unit vector (dx, dy)
    seg          -- ((x1, y1), (x2, y2))
    """
```

You can derive this by solving the 2×2 linear system for the ray parameter t 0 and the segment parameter $u \in [0, 1]$. The starter notebook gives you the formulas if you prefer not to derive them from scratch.

4.3 Step 1.3 — Cast one ray against all segments

Implement:

```
def cast_ray(origin, angle, segments, max_range=10.0):
    """Return the closest intersection distance, clipped to max_range."""
```

Sanity-check it from $(2, 2)$ at `angle = 0` — you should get the distance to the nearest east-facing wall.

4.4 Step 1.4 — Full 360° scan

Sweep $N = 360$ bearings uniformly over $[0, 2\pi)$. Return two arrays `angles` (shape $(N,)$) and `ranges` (shape $(N,)$). Plot:

1. The world (segments + robot).
2. The scan endpoints in world frame (convert from polar using the robot's heading).
3. A faint line from the robot to each endpoint so the beam fan is visible.

Check: the scan endpoints should trace the outline of the walls and the obstacle.

5 Part 2: From One Scan to an Occupancy Grid (30 min)

5.1 Step 2.1 — World grid helpers

Set up a grid covering $x \in [0, 10]$, $y \in [0, 10]$ at $\text{res} = 0.1$ m (so the grid is 100×100). Store it as an (M, N) float array of log-odds, initialized to zero.

Implement `world_to_grid(x, y, origin, res)` and `grid_to_world(i, j, origin, res)`. Round-trip a few points to make sure it works.

5.2 Step 2.2 — Rasterize one ray

Use `skimage.draw.line`:

```
from skimage.draw import line
ii, jj = line(robot_i, robot_j, end_i, end_j)
```

`ii, jj` are arrays of grid indices along the beam. Treat all cells except the last as **free evidence** (+1_free) and the last cell as **occupied evidence** (+1_occ).

5.3 Step 2.3 — Update the grid with one full scan

Wrap everything into:

```
def update_grid(grid, origin, res, pose, angles, ranges,
                l_free=-0.4, l_occ=+0.85, l_clamp=5.0):
    """Update a log-odds grid with one LiDAR scan from `pose = (xR, yR, theta)`.
    Uses Bresenham rasterization per ray."""
```

Run one scan from (2, 2, 0) and plot the resulting probability map:

$$p_{ij} = \frac{1}{1 + e^{-l_{ij}}}.$$

Use `plt.imshow(p.T, origin='lower', cmap='gray_r', vmin=0, vmax=1)` so gray = unknown, white = free, black = occupied. You should see a lit wedge around the robot and a large gray unknown region behind the obstacle.

5.4 Step 2.4 — Short-answer question

Why does the region behind the L-shaped obstacle stay gray after a single scan?
Write one sentence in the notebook.

6 Part 3: Building the Map from Multiple Poses (30 min)

6.1 Step 3.1 — Drive the robot

You are given a list of four poses that cover the room:

```
poses = [  
    (2.0, 2.0, 0.0),  
    (2.0, 8.0, -np.pi / 2),  
    (8.0, 8.0, -np.pi),  
    (8.0, 2.0, +np.pi / 2),  
]
```

At each pose, simulate a full 360° scan and call `update_grid` on the **same** grid (do not reset it). Plot the occupancy grid after each scan so you can watch the unknown region shrink.

6.2 Step 3.2 — Final map

After all four scans, plot:

- The log-odds grid (`cmap='RdBu_r'`, `vmin=-5`, `vmax=5`).
- The probability grid (`cmap='gray_r'`, `vmin=0`, `vmax=1`).

Overlay the robot poses and the true obstacle segments. Confirm that the map recovers the walls.

6.3 Step 3.3 — Effect of noise

Add a small Gaussian noise term to each range (`sigma = 0.05 m`) and rebuild the map. Compare with the noise-free map. Which cells are most affected, and why? Write 2 sentences.

6.4 Step 3.4 — Discussion question

Suppose you halved the grid resolution from 0.1 m to 0.05 m. What changes about (a) memory use, (b) the sharpness of wall edges, (c) the number of rays needed to keep coverage good? One line each.

7 Part 4: A Taste of Vision with OpenCV (30 min)

For this part we use a synthetic “camera image” generated in the notebook — no real camera is needed. The image shows a simulated indoor scene with a red

cone, a blue box, and a gray floor.

7.1 Step 4.1 — Load and inspect

Run the helper that builds the image as a NumPy array, then display it with Matplotlib (`plt.imshow`). Note that OpenCV uses **BGR**, not **RGB** — the helper already uses that ordering.

7.2 Step 4.2 — Convert to HSV

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Threshold for red:

```
lower = np.array([0, 120, 70])
upper = np.array([10, 255, 255])
mask = cv2.inRange(hsv, lower, upper)
```

Show the mask next to the original.

7.3 Step 4.3 — Find the cone

Use `cv2.findContours` on the mask, pick the largest contour by `cv2.contourArea`, and compute its bounding-box centroid. Overlay the box and the centroid on the original image.

7.4 Step 4.4 — Convert a pixel to a steering decision

Split the image into three vertical thirds — left, center, right. Print whether the robot should **turn left**, **go straight**, or **turn right** to face the cone. This is the same logic we saw on the lecture slide for vision-based reactive control.

7.5 Step 4.5 — Compare with LiDAR

In one or two sentences, say what each sensor gave you that the other did not:

- The LiDAR of Parts 1–3 told us *what* about the scene?
 - The camera of Part 4 told us *what* about the scene?
-

8 Wrap-Up

At the end of the lab:

1. Save your notebook with all plots rendered.
2. Upload the `.ipynb` to the course page.

3. In the last cell, write 2-3 sentences on what surprised you — especially about occluded regions, log-odds behavior, or the LiDAR/camera contrast.
-

9 Reference

LiDAR polar $\rightarrow$ Cartesian (sensor frame): $x = r \cos \theta$, $y = r \sin \theta$.

Sensor $\rightarrow$ world: rotate by robot heading θ , then translate by robot position (x_R, y_R) .

World $\rightarrow$ grid: $i = \text{floor}((x - x_{\min}) / \text{res})$, $j = \text{floor}((y - y_{\min}) / \text{res})$.

Log-odds update: $l \leftarrow l + l_{\text{free}}$ along the ray, $l \leftarrow l + l_{\text{occ}}$ at the endpoint. Clamp to $[-5, +5]$.

Log-odds $\rightarrow$ probability: $p = 1 / (1 + \exp(-l))$.

OpenCV pipeline: `imread  $\rightarrow$  cvtColor(BGR2HSV)  $\rightarrow$  inRange  $\rightarrow$  findContours  $\rightarrow$  boundingRect`.